

รายงานผลการปฏิบัติการ Lab 10: Spring WebFlux & WebClient

วิชา CP353002 Software Design and Development | รหัสนักศึกษา: 673380301-0 (Section 1)

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

- **การทำงานของเธรด** : Blocking I/O ใช้โมเดล 1 Thread ต่อ 1 Request โดยเธรดจะต้องหยุดรอ (Block) เมื่อมีคำสั่ง I/O ทำให้สิ้นเปลืองทรัพยากร ในขณะที่ Reactive Programming เป็น Asynchronous Non-blocking ใช้เธรดจำนวนน้อย (Netty Event Loop) โดยเธรดจะไม่หยุดรอ แต่จะปล่อยให้ I/O ทำงานในเบื้องหลังและตอบสนองตาม Event เมื่อข้อมูลพร้อม
- **ความสามารถในการ Scalability** : Blocking รองรับ Concurrent Requests ได้จำกัดตามจำนวน Thread Pool ของเซิร์ฟเวอร์ ส่วน Reactive สามารถรองรับคำขอพร้อมกันได้ปริมาณมหาศาล โดยใช้หน่วยความจำและ CPU น้อยกว่าอย่างเห็นได้ชัด
- **ชนิดข้อมูลที่คืนค่า (Return Type)**: Blocking คืนค่าข้อมูลอ็อบเจกต์โดยตรง เช่น Product หรือ List<Product> ส่วน Reactive คืนค่าเป็น Reactive Stream Publisher ได้แก่ Mono<T> หรือ Flux<T>
- **สัญญาณใน Reactive Streams** : มี 3 สัญญาณหลักคือ 1) onNext(T) ส่งข้อมูลที่แต่ละตัว 2) onError(e) สตรีมหยุดการทำงานเมื่อเกิดข้อผิดพลาด 3) onComplete() สตรีมทำงานเสร็จสมบูรณ์ โดยมีกฎสำคัญคือ "ไม่มีอะไรเกิดขึ้นจนกว่าจะมีการ subscribe"

2. Mono vs Flux — ใช้กรณีไหน และทำไม

- **Mono<T> (คืนค่า 0 หรือ 1 ค่า)**: ใช้กับกรณีที่ทราบแน่นอนว่าผลลัพธ์มีไม่เกิน 1 รายการ เช่น การค้นหาตามรหัส findById(), การบันทึก save(), การอัปเดต update(), หรือการลบข้อมูล (คืน Mono<Void>) เหตุผลที่เลือกใช้เพื่อความชัดเจนของ Method Signature ป้องกัน Overhead ของการจัดการ Collection และมี Operator เฉพาะ เช่น switchIfEmpty()
- **Flux<T> (คืนค่า 0 ถึง N ค่า)**: ใช้กับการดึงชุดข้อมูลหลายรายการ เช่น findAll(), การกรองตามหมวดหมู่ findByCategory(), หรือการสตรีมข้อมูลแบบ Real-time (Event Stream) เหตุผลที่เลือกใช้เพื่อรองรับการทยอยส่งข้อมูลทีละชิ้นแบบ Streaming และมีระบบ Backpressure ช่วยให้ผู้รับสามารถควบคุมอัตราการส่งข้อมูลได้

3. WebClient — อธิบาย Method Chain

WebClient เป็น Non-blocking HTTP Client ของ Spring WebFlux โดยมีการเรียกใช้งานแบบ Method Chain ดังนี้:

```
Mono<Product> product = client.get()           // 1. กำหนด HTTP Method
    .uri("/products/{id}", "1")                 // 2. ระบุ Endpoint URL
    .retrieve()                                 // 3. ยิงคำขอและรับ response
    .bodyToMono(Product.class);                 // 4. แปลงข้อมูลเป็น Mono<Product>
```

- **1) .get() / .post() / .delete()**: กำหนด HTTP Method ที่ต้องการส่งคำขอไปยัง API ปลายทาง
- **2) .uri(...)**: กำหนด URI Endpoint ปลายทาง พร้อมแทรก Path Variable หรือ Query Parameter
- **3) .retrieve()**: ส่งคำขอแบบ Asynchronous Non-blocking และเตรียมถอดรหัส Response Body
- **4) .bodyToMono() / .bodyToFlux()**: แปลง Response Body ให้อยู่ในรูป Reactive Publisher (Mono สำหรับ 1 ค่า หรือ Flux สำหรับหลายค่า)

4. โค้ดสำคัญของ Repository, Service และ Controller

- **ProductRepository.java**: จัดเก็บข้อมูล In-Memory ด้วย ConcurrentHashMap มีการ Seed ข้อมูลตัวอย่างสินค้า iPhone 15 Pro (673380301-0 SEC 1) โดยเมธอด findById() คืน Mono.just(product) เมื่อพบข้อมูลหรือ Mono.empty() เมื่อไม่พบ, เมธอด findAll() ใช้ Flux.fromIterable(), และ findByCategory() ใช้ .filter() กรองข้อมูลตามหมวดหมู่
- **ProductService.java**: จัดการ Business Logic โดยเมธอด getById() ใช้ .switchIfEmpty(Mono.error(...)) เพื่อโยน RuntimeException หากไม่พบสินค้า, เมธอด save() ทำการสร้างรหัส UUID อัตโนมัติหากสินค้ายังไม่มี ID, และ getDiscountedPrice() ใช้ .map(Product::getDiscountedPrice) คำนวณราคาหลังส่วนลด

- **ProductController.java:** สร้าง Reactive REST Controller ด้วย `@RestController` และ `@RequestMapping("/products")` รองรับ 6 Endpoints เพื่อส่งต่อไปยัง Service ได้แก่ GET `/id`, GET `/`, POST `/`, DELETE `/id`, GET `/category/{cat}`, และ GET `/id/price`

5. ผลการทดสอบทุก Endpoint และ Unit Tests

5.1 GET `/products/1` — ดึงข้อมูลสินค้าชิ้นที่ 1 (แสดงรหัสนักศึกษา)

- **URL:** `http://localhost:8080/products/1`
- **ผลลัพธ์ (Response):** `{"id":"1", "name":"iPhone 15 Pro (673380301-0 SEC 1)", "category":"Electronics", "price":39900.0, "discountType":"MEMBER", "discountedPrice":35910.0}`

```
localhost:8080/products/1
Pretty-print ☐

{"id":"1","name":"iPhone 15 Pro (673380301-0 SEC 1)","category":"Electronics","brand":"Apple","stock":50,"price":39900.0,"discountType":"MEMBER","discountedPrice":35910.0}
```

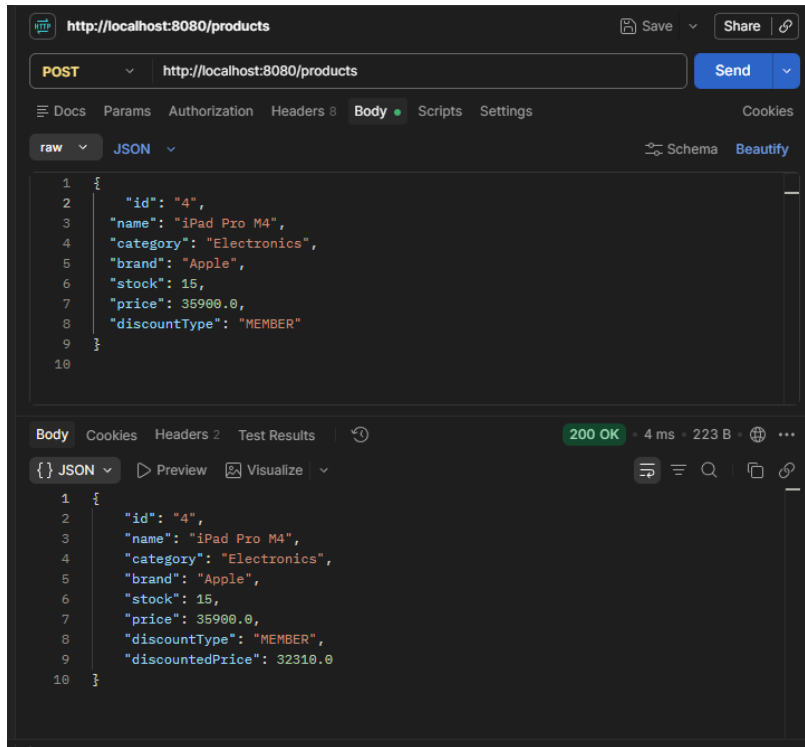
5.2 GET `/products` — ดึงรายการสินค้าทั้งหมด (Flux<Product>)

- **URL:** `http://localhost:8080/products`
- **ผลลัพธ์ (Response):** คืน Array ของสินค้าทั้งหมด 3 รายการ (iPhone 15 Pro, MacBook Air M3, Samsung Galaxy S24)

```
[{"id":"1","name":"iPhone 15 Pro (673380301-0 SEC 1)","category":"Electronics","brand":"Apple","stock":50,"price":39900.0,"discountType":"MEMBER","discountedPrice":35910.0}, {"id":"2","name":"MacBook Air M3","category":"Electronics","brand":"Apple","stock":20,"price":49900.0,"discountType":"NONE","discountedPrice":49900.0}, {"id":"3","name":"Samsung Galaxy S24","category":"Electronics","brand":"Samsung","stock":30,"price":29900.0,"discountType":"SEASONAL","discountedPrice":23920.0}]
```

5.3 POST /products — เพิ่มข้อมูลสินค้าใหม่

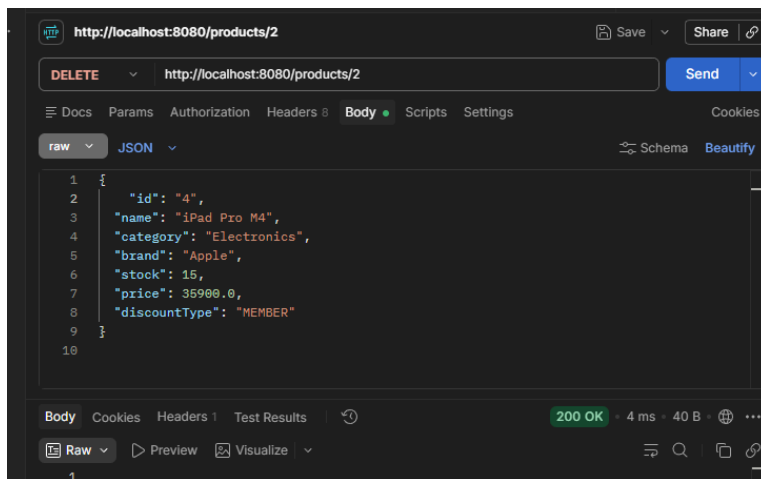
- **URL:** http://localhost:8080/products (Method: POST)
- **Request Body:** {"name":"iPad Pro M4", "category":"Electronics", "price":35900.0, "discountType":"MEMBER"}
- **ผลลัพธ์ (Response):** คืน Object สินค้าที่บันทึกแล้วพร้อมรหัส ID ที่ระบบสร้างขึ้นอัตโนมัติ



```
[{"id":"1","name":"iPhone 15 Pro (673380301-0 SEC 1)","category":"Electronics","brand":"Apple","stock":50,"price":39900.0,"discountType":"MEMBER","discountedPrice":35910.0}, {"id":"2","name":"MacBook Air M3","category":"Electronics","brand":"Apple","stock":20,"price":49900.0,"discountType":"NONE","discountedPrice":49900.0}, {"id":"3","name":"Samsung Galaxy S24","category":"Electronics","brand":"Samsung","stock":30,"price":29900.0,"discountType":"SEASONAL","discountedPrice":23920.0}, {"id":"4","name":"iPad Pro M4","category":"Electronics","brand":"Apple","stock":15,"price":35900.0,"discountType":"MEMBER","discountedPrice":32310.0}]
```

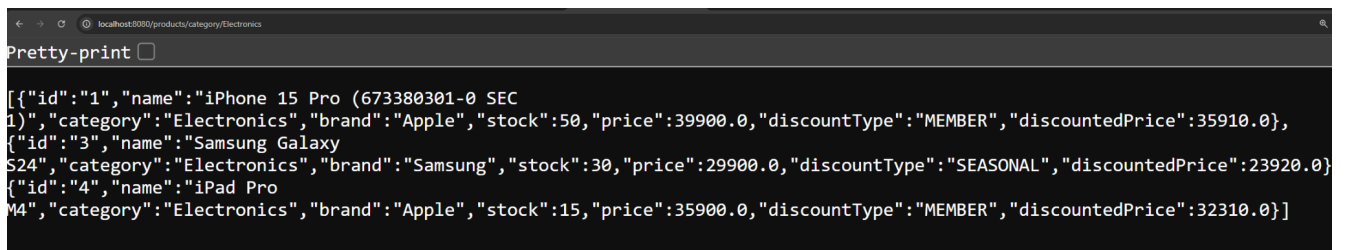
5.4 DELETE /products/2 — ลบข้อมูลสินค้าตาม ID

- **URL:** http://localhost:8080/products/2 (Method: DELETE)
- **ผลลัพธ์ (Response):** HTTP Status 200 OK (คืน Mono<Void> ไม่มี Body)



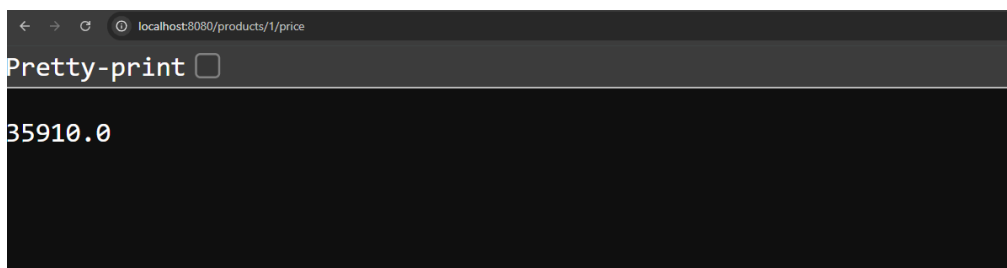
5.5 GET /products/category/Electronics — กรองสินค้าตามหมวดหมู่

- URL: <http://localhost:8080/products/category/Electronics>
- ผลลัพธ์ (Response): คืนรายการสินค้าที่อยู่ในหมวดหมู่ Electronics ทั้งหมด



5.6 GET /products/1/price — ดึงราคาลงหักส่วนลด

- URL: <http://localhost:8080/products/1/price>
- ผลลัพธ์ (Response): 35910.0 (คำนวณหักส่วนลดประเภท MEMBER 10% จากราคา 39,900 บาท)



5.7 ผลการรัน Unit Tests (mvn test)

- คำสั่งทดสอบ: mvn test
- ผลลัพธ์: Tests run: 6, Failures: 0, Errors: 0, Skipped: 0 — BUILD SUCCESS

```
java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.060 s -- in com.example.lab10.Lab10ApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.970 s
[INFO] Finished at: 2026-09-22T02:17:45+07:00
[INFO]
PS C:\Users\tanku\Documents\GitHub\claude-code-main\673380301-0-LAB10_WebClient>
loading source jar from known Maven repositories for local repository file. ☞ Java: Ready
```